
PPO and TD3 for Safe Reinforcement Learning Using Control Barrier Functions

David Fornos Fernandez
Department of Aerospace Engineering
Texas A&M University
College Station, TX 77843
dfornos@tamu.edu

Abhijeet
Department of Aerospace Engineering
Texas A&M University
College Station, TX 77843
abhinir@tamu.edu

Abstract

In this project, we apply reinforcement learning to determine control inputs that optimize the performance of a dynamical system. We adopt a reinforcement learning framework in which an agent learns policies that accomplish a target task while remaining inside a prescribed safe set enforced by Control Barrier Functions. Two algorithms—Proximal Policy Optimization (PPO) and Twin-Delayed DDPG (TD3)—are implemented and evaluated against the Soft Actor-Critic (SAC) baseline. Experiments on a unicycle robot illustrate the method: the trained policies consistently steer the vehicle to the goal while respecting state constraints, and successfully generalize to novel obstacle layouts.

1 Introduction

Ensuring the safe operation of a dynamical system remains a significant challenge in control design. Consider, for example, a car-like robot tasked with navigating from an initial position to a destination while avoiding static and dynamic obstacles along its path. Or imagine a robotic vacuum cleaner that must efficiently cover the floor area of a home without colliding with furniture such as sofas, chairs, or tables. Similar challenges arise in a wide range of applications: a drone performing infrastructure inspection must maintain a safe distance from power lines, and an underwater autonomous vehicle must maneuver through narrow passages without contacting the surrounding environment. In all these cases, the goal is not only to accomplish a specific task but to do so while respecting strict operational constraints. In fact, problems like these are very common where one needs to steer a dynamical system under a constrained environment.

Traditional control methods often struggle to handle such hard safety requirements systematically, motivating the development of more formal techniques like Control Barrier Functions. Control Barrier Functions (CBFs), first proposed by [2], offer a principled way to encode those constraints. By embedding a barrier term in the control or learning loop, one can guarantee that the policy stays within the safe set while still pursuing its performance objective. The approach is extended to apply to dynamic walking of a robot on stepping stairs and dynamic balancing on Segways, which are quite complicated tasks [1]. Control Barrier Functions provide a real-time, optimization-based safety layer that projects a tentative action onto the closest safe action via a quadratic program (QP) [2]. Early demonstrations applied CBFs to robotic locomotion and multi-robot collision avoidance [12].

2 Related Work

As the concept of Control Barrier Functions (CBFs) gained traction, researchers began integrating them with reinforcement learning frameworks to enhance safety guarantees. Several Safe-RL approaches have addressed related challenges. For instance, Cheng *et al.* [3] were among the first

to embed a CBF-QP inside a policy-gradient loop, showing safe learning on car-following and inverted-pendulum tasks. Emam *et al.* [4] extended that idea by making the QP *differentiable*, allowing gradients to flow through the safety filter; they also introduced *Robust* CBFs that account for GP-modeled disturbances. More recent work exploits logarithmic or polynomial barrier penalties [9] and cascaded high-relative-degree CBF designs for quadrotors and PVTOL aircraft [4].

In many practical applications, access to a system model makes *model-based reinforcement learning* a natural and effective extension for safe learning with CBFs. Model-based approaches improve sample efficiency by enabling short simulated roll-outs under an approximate dynamics model. Fisac *et al.* [5] combined Hamilton–Jacobi reachability analysis with model predictive control (MPC) to ensure safe exploration during learning. Janner *et al.* [8] demonstrated that short-horizon model roll-outs can significantly stabilize value function training; subsequent work [4] showed that integrating these roll-outs with a CBF layer further accelerates safe policy learning.

While most CBF-based reinforcement learning studies have concentrated on the Soft Actor-Critic (SAC) algorithm [7], the performance of Proximal Policy Optimization (PPO) [11] and Twin-Delayed DDPG (TD3) [6] in this setting remains relatively unexplored. Additionally, there has been limited investigation into how well policies trained with CBFs generalize across different obstacle environments. In this work, we address both gaps by implementing PPO and TD3 within a CBF framework and systematically benchmarking them against SAC. Specifically, we train each agent on a unicycle robot navigating through one obstacle field and evaluate their ability to generalize by testing on a distinct, unseen obstacle configuration, all while maintaining safety through the CBF constraints.

The remainder of the paper is organized as follows. Section 3 formalizes the problem setup. Section 4 details our PPO and TD3 implementations, and Section 5 presents convergence curves, per-episode training times, and trajectory plots that illustrate obstacle avoidance in two different fields. Finally, we provide our conclusions and possible directions for future work.

3 Problem Formulation

In this work, we aim to design safe reinforcement learning controllers for a unicycle robot navigating through an obstacle field while reaching a target goal. We formalize the underlying system dynamics, the safety constraints via robust control barrier functions (RCBFs), the reward structure, and the modifications adopted for sample efficiency.

3.1 Disturbed System Dynamics

The unicycle dynamics with unknown disturbances are modeled as a continuous-time, control-affine system:

$$\dot{x}(t) = f(x(t)) + g(x(t))u(t) + d(x(t)), \quad (1)$$

where

- $x = [p_x, p_y, \theta]^T \in \mathbb{R}^3$ is the robot state,
- $u = [v, \omega]^T \in \mathbb{R}^2$ is the control input (linear and angular velocity),
- $d(x) \in \mathbb{R}^3$ denotes an unknown disturbance term.

The nominal drift and control matrices are:

$$f(x) = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \quad g(x) = \begin{bmatrix} \cos \theta & 0 \\ \sin \theta & 0 \\ 0 & 1 \end{bmatrix}.$$

In discrete time, using a sampling interval Δt , the dynamics are approximated as:

$$x_{k+1} = x_k + \Delta t (f(x_k) + g(x_k)u_k + d(x_k)). \quad (2)$$

3.2 Control Barrier Functions for Safety

To guarantee safety, we enforce that the robot state remains inside a safe set:

$$\mathcal{C} = \{x \in \mathbb{R}^3 \mid h(x) \geq 0\},$$

where $h : \mathbb{R}^3 \rightarrow \mathbb{R}$ is a continuously differentiable function associated with safety constraints.

Nominal CBF Condition: For the nominal (undisturbed) system, safety is guaranteed if:

$$\dot{h}(x, u) + \gamma h(x) \geq 0,$$

for some $\gamma > 0$, where

$$\dot{h}(x, u) = \nabla h(x)^\top (f(x) + g(x)u).$$

Robust CBF Condition: In the presence of unknown disturbance $d(x)$, we use a Robust Control Barrier Function (RCBF) that satisfies:

$$\inf_{d \in \mathcal{D}(x)} (\nabla h(x)^\top (f(x) + g(x)u + d)) + \gamma h(x) \geq 0,$$

where $\mathcal{D}(x)$ denotes the learned uncertainty set from a Gaussian Process (GP) model.

3.3 Safe Action via Quadratic Programming

At every time step, the agent's proposed action u_{des} is adjusted by solving the following Quadratic Program:

$$\begin{aligned} u^* = \arg \min_{u \in \mathbb{R}^m} \quad & \|u - u_{\text{des}}\|^2 \\ \text{s.t.} \quad & \dot{h}_i(x, u) + \gamma h_i(x) \geq 0, \quad \forall i. \end{aligned} \quad (3)$$

The resulting safe control u^* is applied to the robot.

3.4 Reward Structure

In most literature in optimization, a quadratic cost function penalizing state and control deviations was used:

$$r(x, u) = -(x^\top Q x + u^\top R u),$$

where $Q \succ 0, R \succ 0$ are positive definite matrices.

For simplicity and faster learning, in this project we adopt a reward structure based on the *change in distance to the goal*:

$$r_k = d(x_k) - d(x_{k+1}), \quad (4)$$

where $d(x)$ is the Euclidean distance to the goal location.

- Positive reward is given for reducing distance to the goal, - Additional terminal reward of +1 is given when the goal is reached within a threshold radius.

3.5 Unicycle Environment and Obstacle Fields

The environment consists of:

- A bounded 2D plane with Cartesian coordinates.
- A set of static circular obstacles of radius 0.6 meters placed at fixed positions during training.
- A separate obstacle field during testing with different obstacle positions and configurations.

Figure 1 illustrates the obstacle field used to train reinforcement learning agents for the unicycle environment. The unicycle begins near the initial position $(-2.5, -2.5)$, subject to slight uncertainty, and is required to reach a goal region of radius 0.3 centered at $(2.5, 2.5)$.

Collision avoidance is enforced through CBF constraints for each obstacle:

$$h_i(x) = \|p(x) - p_i\|^2 - r_{\text{safe}}^2,$$

where $p(x) = (p_x, p_y)$ is the robot's position, p_i is the center of the i -th obstacle, and r_{safe} is the minimum allowed clearance. There is a cost incurred everytime the agent goes in the obstacle field where the cost is given by:

$$c_k = 0.1.$$

So, the total cost for any trajectory will be 0.1 times the number of steps at which the agent is inside the obstacle field.

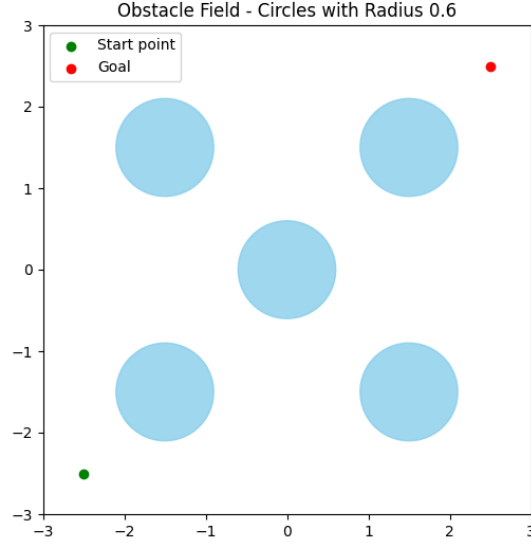


Figure 1: The obstacle field for the unicycle environment with goal and one of the starting points used for training.

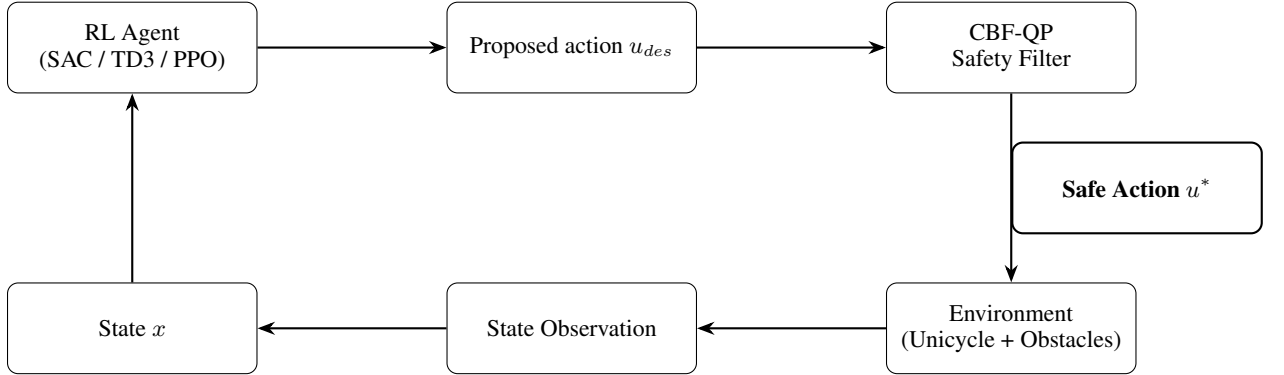


Figure 2: Architecture of Safe RL with CBF-QP filtering. The RL agent proposes an action, which is filtered to ensure safety and then applied to the environment. The observed state is fed back to the agent.

4 Research Approach

As mentioned in the introduction, in this paper we implement two Reinforcement Learning algorithms to tackle the safe reinforcement learning problem. These are Proximal Policy Approximation (PPO) and Twin-Delayed Deep Deterministic Policy Gradient (TD3).

4.1 Proximal Policy Approximation (PPO)

The implementation combines the advantages of PPO’s clipped surrogate objective with Control Barrier Functions (CBF) for safety guarantees.

4.1.1 Network Architecture

The PPO implementation consists of two main neural networks:

- **Policy Network** (π_θ): A Gaussian policy network that outputs mean and standard deviation for action selection:

$$\pi_\theta(a|s) = \mathcal{N}(\mu_\theta(s), \sigma_\theta(s)) \quad (5)$$

- **Value Network** (V_ϕ): A value network that estimates the state-value function:

$$V_\phi(s) \approx \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t | s_0 = s \right] \quad (6)$$

4.1.2 PPO Algorithm

The core of our implementation is the PPO algorithm with clipped surrogate objective. The control loop is presented in Algorithm 1.

Algorithm 1 Proximal Policy Optimization with CBF

- 1: Initialize policy parameters θ_0 and value function parameters ϕ_0
- 2: **for** iteration $k = 0, 1, 2, \dots$ **do**
- 3: **for** episode $e = 1, 2, \dots, N$ **do**
- 4: Collect trajectory $\tau = (s_0, a_0, r_0, \dots, s_T)$
- 5: Compute advantages $\hat{A}_t$ using GAE
- 6: Compute value targets $V_{target}(s_t) = \hat{A}_t + V_{\phi_k}(s_t)$
- 7: Update policy by maximizing the PPO-clip objective:

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t [\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)] \quad (7)$$

- 8: Update value function by regression on mean-squared error:

$$L^{VF}(\phi) = \hat{\mathbb{E}}_t [(V_\phi(s_t) - V_{target}(s_t))^2] \quad (8)$$

- 9: **end for**
 - 10: **end for**
-

The implementation of our PPO algorithm incorporates several key features that were selected to balance learning efficiency with stability. Each component was chosen based on empirical evidence and theoretical foundations found in the literature, with particular attention to the requirements of robotic control tasks.

4.1.3 Implementation Details

First of all, we used Generalized Advantage Estimation (GAE) [10]. The GAE provides a balance between bias and variance in advantage estimation through the parameter λ , which we set to 0.95. This value was selected to favor low variance while maintaining reasonable bias, as higher values can lead to unstable learning. The GAE is computed as:

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l} \quad (9)$$

where $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ represents the temporal difference error. This formulation allows for efficient computation of advantages using a single backward pass through the trajectory, while the $\gamma\lambda$ term ensures proper credit assignment across time steps. The temporal difference error provides immediate feedback about the quality of actions, while the discounted sum ensures that long-term consequences are properly considered.

The clipped surrogate objective forms the core of our policy update mechanism. The policy update ratio $r_t(\theta)$ is defined as:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \quad (10)$$

We implemented this with a conservative clipping parameter $\epsilon = 0.1$, which is more restrictive than the standard value of 0.2. This choice was motivated by the need to prevent large policy updates that could destabilize learning, particularly in the context of robotic control where safety is very important. The ratio $r_t(\theta)$ provides a measure of policy change while maintaining the original policy’s behavior, and the clipping mechanism ensures that the policy update stays within a trust region.

To encourage exploration and prevent premature convergence to deterministic policies, we incorporate an entropy bonus term:

$$L^{ENT}(\theta) = \beta \hat{\mathbb{E}}_t[\mathcal{H}(\pi_\theta(\cdot|s_t))] \quad (11)$$

The entropy coefficient $\beta = 0.01$ was chosen to provide sufficient exploration while not overwhelming the main objective. The entropy term $\mathcal{H}(\pi_\theta(\cdot|s_t))$ encourages the policy to maintain stochasticity, which is particularly important in our task where the agent needs to explore different strategies to reach the goal while avoiding obstacles. This approach helps maintain a balance between exploration and exploitation throughout training, preventing the policy from becoming too deterministic too early in the learning process.

The value function is updated using mean-squared error loss, providing stable value estimates. We employ a separate value network rather than a shared network to prevent interference between policy and value updates. The value function learning rate matches the policy learning rate to maintain consistent update scales, and gradient clipping is applied to both networks to prevent large parameter updates. This separation of concerns between policy and value networks has been shown to improve learning stability in complex environments.

For the training process, we use mini-batches of size 256, which provides sufficient samples for stable updates while maintaining computational efficiency. We perform 10 epochs of updates per batch to ensure thorough learning while preventing overfitting. The learning rate of 3×10^{-4} was selected through empirical testing to provide stable learning without being too conservative.

4.1.4 Hyperparameters

The key hyperparameters used in our implementation are:

Table 1: PPO Hyperparameters

Parameter	Value
Learning Rate (α)	3×10^{-4}
Discount Factor (γ)	0.99
GAE Parameter (λ)	0.95
Clipping Parameter (ϵ)	0.1
Entropy Coefficient (β)	0.01
Batch Size	256
Epochs per Update	10

4.2 Twin-Delayed Deep Deterministic Policy Gradient with Control Barrier Functions

We also implement a safe version of Twin-Delayed Deep Deterministic Policy Gradient (TD3) using Control Barrier Functions (CBFs) to guarantee state constraint satisfaction. TD3 [6] improves over DDPG by addressing overestimation bias in Q-learning through the use of:

- Two critic networks (Q_1, Q_2) with target minimization,
- Delayed actor updates,
- Target policy smoothing by adding clipped noise.

Given a batch of transitions $(s, a, r, s', \text{done})$ from a replay buffer, the target value is computed as:

$$y = r + \gamma(1 - \text{done}) \min(Q'_1(s', a'_{\text{target}}), Q'_2(s', a'_{\text{target}})), \quad (12)$$

where a'_{target} is the next action with clipped noise:

$$a'_{\text{target}} = \text{clip}(\pi'(s') + \epsilon, -c, c), \quad \epsilon \sim \mathcal{N}(0, \sigma^2).$$

The critics are updated by minimizing the Bellman loss:

$$\mathcal{L} = \frac{1}{N} \sum (Q_i(s, a) - y)^2, \quad i = 1, 2,$$

and the actor is updated less frequently to stabilize training:

$$\nabla_{\theta} J(\pi) = \frac{1}{N} \sum \nabla_a Q_1(s, a)|_{a=\pi(s)} \nabla_{\theta} \pi(s).$$

In our implementation, each action proposed by the TD3 policy is passed through a differentiable CBF-QP layer to enforce safety before being applied to the environment.

At each step:

1. The agent proposes an action u_{des} from its deterministic policy.
2. The CBF-QP solves the following optimization:

$$\begin{aligned} u^* = \arg \min_u \quad & \|u - u_{\text{des}}\|^2 \\ \text{s.t.} \quad & \nabla h_i(x)^\top (f(x) + g(x)u) + \gamma h_i(x) \geq 0, \quad \forall i, \end{aligned} \quad (13)$$

yielding a safe action u^* that is guaranteed to maintain safety.

We follow a model-based reinforcement learning (MBRL) approach:

- A Gaussian Process (GP) disturbance model predicts uncertainty.
- Short simulated rollouts supplement real transitions, improving sample efficiency.
- Safety is enforced during model rollouts using the same CBF-QP layer.

Table 2: TD3 Training Hyperparameters

Hyperparameter	Value
Policy noise standard deviation σ	0.2
Noise clip threshold c	0.5
Policy update frequency	Every 2 critic updates
Replay buffer size	10^6 transitions
Batch size	256
Learning rate (actor and critics)	10^{-3}

The agent is trained on a randomly initialized unicycle environment populated with circular obstacles and validated on an unseen field.

5 Results

In order to assess the performance of the proposed methods, we evaluate our results by computing the same critical metrics adopted in [4], where Soft Actor-Critic (SAC) was employed as a baseline for safe reinforcement learning. This allows us to perform a meaningful and robust benchmarking.

The environment used in our experiments is identical to the one described in [4], specifically the Unicycle environment. Our implementation builds directly on the publicly available codebase provided by the authors at <https://github.com/yemam3/Mod-RL-RCBF>. The hyperparameters used in our PPO implementation are listed in Tables 1 and 2. For SAC, we adopted the exact hyperparameter values specified in the original paper and available in the reference implementation.

It is important to highlight that, for a consistent and statistically significant evaluation, all results are averaged across **15 independent training runs** per algorithm. Each run spans **200 episodes**, with each episode comprising **1000 environment steps**. In this context, such an extensive evaluation ensures that random fluctuations are effectively averaged out, in order to offer reliable insights into both average behavior and variability across different random seeds.

As previously mentioned, a one of the most important elements of the evaluation is the evolution of the episodic reward over training. In Figure 3, we present the average reward across episodes for PPO, SAC, and TD3, with shaded regions representing one standard deviation across runs.

SAC improves the fastest during the early episodes, reaching moderate reward levels quickly. However, its performance levels off and does not increase much after about 25 episodes, even declining towards the end. PPO improves more slowly during the initial stages but keeps increasing its reward across training. By the end of the episodes, PPO reaches the highest reward among the three algorithms, with relatively low variability across runs. TD3 shows faster early improvement than PPO but reaches a lower final reward, staying below both SAC and PPO for most of the training. Therefore, PPO achieves the best final reward, while SAC and TD3 both converge earlier but at lower reward values. Regarding variability, TD3 seems to be the most reliable, since its shaded area is the narrowest among the algorithms. PPO has also a relatively small variation and SAC ended up with the widest variability across runs, which was somewhat unexpected given how consistent it was at the beginning. One possible reason is that SAC maintains a high level of exploration throughout training due to its entropy regularization. While this helps early learning, it can introduce instability later on if the policy doesn't become more focused or if entropy isn't annealed.

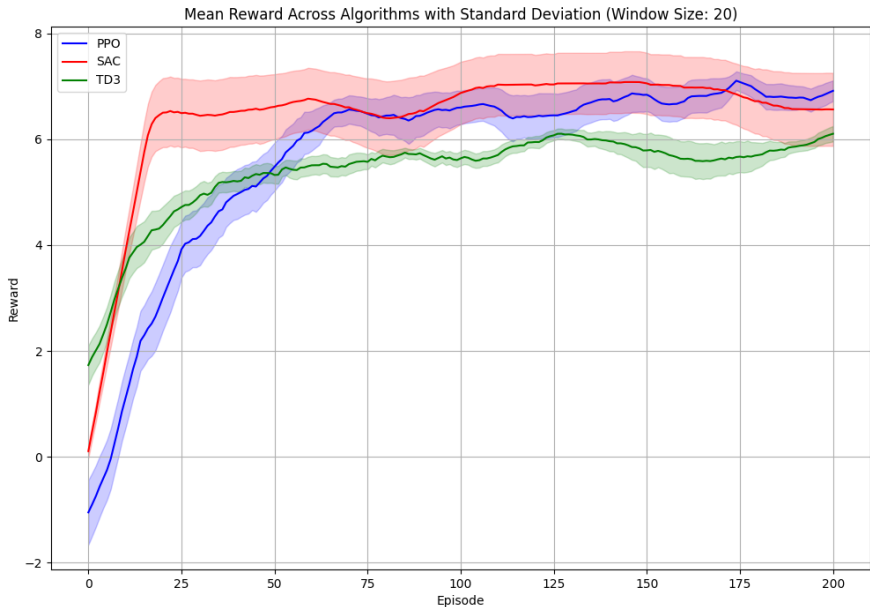


Figure 3: Episode reward versus training episode for PPO, TD3, and SAC, averaged over 15 runs. The shaded region corresponds to one standard deviation.

Another important factor, from a broader perspective, is the actual computational effort required by each algorithm. Essentially, even if two algorithms achieve similar reward levels, the training runtime can determine their practical viability. Therefore, in Figure 4, we include boxplots comparing the average per-episode runtime for PPO, TD3, and SAC. This visualization captures the remarkable differences in computational efficiency among the methods. As seen in the figure, TD3 consistently achieves the fastest runtimes per episode, which aligns with its lightweight architecture and less frequent policy updates compared to PPO and SAC. Notably, SAC exhibits significantly longer episode times, which highlights its computationally intensive nature due to the multiple critic updates

and entropy tuning steps. At the end of the day, this substantial gap in runtime is critical for practitioners who must balance learning performance with resource constraints.

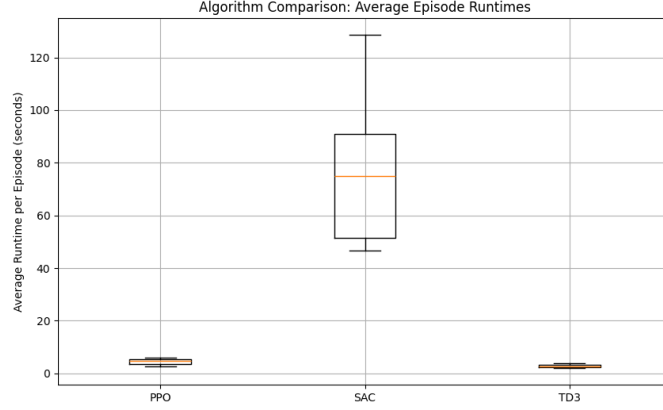


Figure 4: Boxplots of average episode runtimes for PPO, TD3, and SAC, across 15 training runs.

Finally, to qualitatively assess the success of the policies beyond numerical rewards, we generated and evaluated trajectory plots of the agent navigating the obstacle field. Figure 5 shows examples of the trajectories followed by the unicycle robot after training with PPO, TD3 and SAC, both in the original 5 circles field used for training and in a novel obstacle configuration unseen during training, which consists of one triangle, one circle and one square. It is important to highlight that these trajectory plots demonstrate the agents’ ability to generalize and avoid obstacles safely. This confirms that CBFs were effective in keeping agents within constraints. Furthermore, it is interesting to note the smoothness and decisiveness of the paths generated by TD3, whereas PPO occasionally exhibits slightly more circuitous trajectories, possibly due to its stochastic policy updates. In any case, the safe navigation achieved across all test scenarios represents a remarkable outcome.

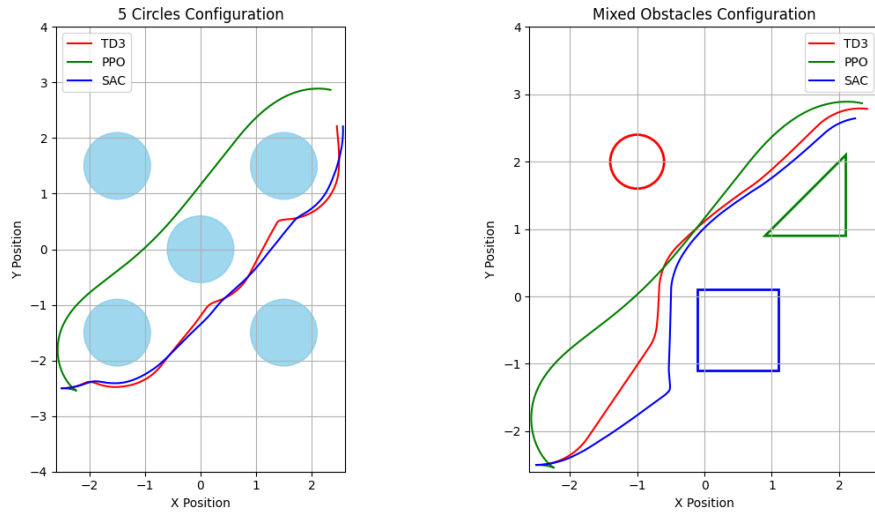


Figure 5: Example trajectories of the trained agents navigating obstacle fields. Left: original environment; Right: unseen obstacle configuration.

6 Conclusions and Future work

6.1 Discussion

In this project, we developed safe reinforcement learning controllers for a unicycle robot using Proximal Policy Optimization (PPO) and Twin-Delayed Deep Deterministic Policy Gradient (TD3), and compared them against a Soft Actor-Critic (SAC) baseline. All agents were trained to navigate from random initial positions to a goal while avoiding obstacles, with safety guaranteed through Control Barrier Functions (CBFs).

Our results demonstrate that all three agents successfully learned safe policies. In terms of training dynamics, SAC exhibited the fastest initial reward improvement but plateaued early, while PPO achieved the highest final reward despite a slower start. TD3 showed moderate early improvement and consistent performance but did not reach the same final reward as PPO and SAC.

In terms of runtime efficiency, TD3 was the most computationally lightweight per episode, followed by PPO, with SAC being the slowest—largely due to its off-policy updates and entropy regularization overhead. These results are significant for applications where computational cost is a concern.

Trajectory analysis further confirmed that all agents generalized well to unseen obstacle configurations, with TD3 producing the most direct and smooth paths, and PPO occasionally displaying more circuitous trajectories due to its stochastic nature. Nonetheless, CBFs successfully enforced constraint satisfaction in all cases.

At the end, choosing the best algorithm for safe reinforcement learning with CBFs depends on the specific goals and constraints of the practitioner. If the main priority is to achieve the highest final reward in a safety-critical setting, PPO appears to be a strong choice. For scenarios where quick early learning is important, SAC may be more attractive, although its higher variance should be considered. TD3 offers a good balance between sample efficiency and runtime performance, making it appealing in resource-constrained applications where consistency matters. Each method brings trade-offs, and the most suitable choice depends on factors like training time budget, safety requirements, computational limitations, and the complexity of the control task.

6.2 Future Work

There are several directions we would like to explore after this project. One idea is to integrate a quadratic cost structure into the reward function, together with a logarithmic barrier that penalizes constraint violations more directly. This kind of objective could help produce smoother actions while making constraint handling feel more natural during learning.

We’re also interested in applying this safe reinforcement learning approach to more complex systems. A good next step would be to test it on robotic arms or other platforms with many degrees of freedom. These systems come with different types of challenges—more variables, more coupled dynamics—and would help us understand how well the methods scale in practice.

Another area worth exploring is how to tune the CBF parameters automatically. Right now, selecting these values often requires some trial and error. If the parameters could adapt during training, or even be learned alongside the policy, the framework could become more flexible and easier to apply in new settings.

Lastly, we think it would be helpful to look at how these agents behave when there’s uncertainty in the environment. This includes things like sensor noise, disturbances, or imperfect models. Combining what we’ve done with ideas from robust control might help the agent stay safe and perform well even when the situation changes.

6.3 Learning Outcomes

This project helped us understand some core aspects of reinforcement learning more clearly. Working with both on-policy and off-policy algorithms gave us a sense of how different training paradigms behave under similar conditions. The difference between stochastic and deterministic policies also became more apparent, especially in how they explore and adapt over time.

We also saw how the design of loss functions, the timing of updates, and the use of value estimates all interact to shape the learning process. Concepts like advantage estimation and policy ratios, which might seem abstract at first, made much more sense when we observed how small changes in implementation affected the agent’s performance.

One thing that stood out during evaluation was the need to assess both the final reward and the behavior of the agent. A policy might score well numerically while behaving in ways that are inefficient or unnatural. Watching what the agent actually does, in addition to checking rewards, gave a much better picture of what it had really learned.

This hands-on experience helped us build a stronger intuition for how reinforcement learning works in practice, and how theory connects to real training behavior. It also made clear that many aspects of algorithm design—like update frequency, entropy terms, or network initialization—play a much bigger role than they might seem from just reading the equations.

References

- [1] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada. Control barrier functions: Theory and applications, 2019.
- [2] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada. Control barrier function based quadratic programs for safety critical systems. *IEEE Transactions on Automatic Control*, 62(8):3861–3876, Aug. 2017.
- [3] R. Cheng, G. Orosz, R. M. Murray, and J. W. Burdick. End-to-end safe reinforcement learning through barrier functions for safety-critical continuous control tasks, 2019.
- [4] Y. Emam, G. Notomista, P. Glotfelter, Z. Kira, and M. Egerstedt. Safe reinforcement learning using robust control barrier functions, 2022.
- [5] J. F. Fisac, A. K. Akametalu, M. N. Zeilinger, S. Kaynama, J. Gillula, and C. J. Tomlin. A general safety framework for learning-based control in uncertain robotic systems. *IEEE Transactions on Automatic Control*, 64(7):2737–2752, 2019.
- [6] S. Fujimoto, H. van Hoof, and D. Meger. Addressing function approximation error in actor-critic methods, 2018.
- [7] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, 2018.
- [8] M. Janner, J. Fu, M. Zhang, and S. Levine. When to trust your model: Model-based policy optimization, 2021.
- [9] Z. Marvi and B. Kiumarsi. Safe reinforcement learning: A control barrier function optimization approach. *International Journal of Robust and Nonlinear Control*, 31(6):1923–1940, 2021.
- [10] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.
- [11] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms, 2017.
- [12] L. Wang, A. D. Ames, and M. Egerstedt. Safety barrier certificates for collisions-free multirobot systems. *IEEE Transactions on Robotics*, 33(3):661–674, 2017.